CREDIT CARD APPROVAL PREDICTION USING MACHINE LEARNING

Project Description

The Credit Card Approval Prediction Using Machine Learning project is designed to automate the process of predicting whether a credit card application should be approved or rejected based on an applicant's personal, financial, and employment information. Traditionally, financial institutions evaluate numerous applicant details manually, making the approval process time-consuming and prone to human error. This project addresses that challenge by utilizing Machine Learning algorithms to provide accurate and efficient predictions.

The project uses the Credit Card Approval Prediction dataset obtained from Kaggle, containing applicant demographic and financial information. The dataset undergoes preprocessing, including handling missing values, encoding categorical features, feature scaling, and feature selection. Multiple Machine Learning models, including Logistic Regression, Decision Tree, Random Forest, and XGBoost, are trained and evaluated. After comparing their performance, XGBoost is selected as the final model due to its superior prediction accuracy.

A user-friendly Flask web application has been developed to allow users to enter applicant details through an HTML interface. The trained XGBoost model predicts whether the applicant is likely to receive credit card approval. The application provides an efficient, fast, and reliable decision support system for credit card approval prediction.

Scenarios

Scenario 1: New Credit Card Applicant
A customer applies for a credit card by providing personal, employment, income, and family details. The application processes the information through the trained Machine Learning model and predicts whether the credit card application should be approved or rejected.

Scenario 2: Banking Decision Support
A bank employee enters the applicant's information into the web application. The Machine Learning model evaluates the applicant's profile and instantly provides an approval prediction, reducing manual effort and improving consistency in decision-making.

Scenario 3: Automated Credit Evaluation
Financial institutions can integrate this prediction system into their existing application workflow to automatically evaluate multiple credit card applications with greater speed, consistency, and accuracy while minimizing human intervention.

Technical Architecture:

The Credit Card Approval Prediction system follows a Machine Learning-based architecture that processes applicant information, performs data preprocessing, predicts credit card approval using a trained XGBoost model, and displays the prediction through a Flask web application.
The architecture consists of the following components:

1.Dataset Collection

The project utilizes the Credit Card Approval Prediction dataset obtained from Kaggle. The dataset consists of applicant demographic, employment, income, and financial information used for model training and evaluation.

2. Data Preprocessing

The collected dataset undergoes several preprocessing steps, including:

- Handling missing values
- Removing duplicate records
- Encoding categorical features
- Feature scaling
- Feature selection

These preprocessing techniques improve the quality of data before training the Machine Learning models.

3. Machine Learning Model Development

The preprocessed dataset is divided into training and testing datasets. Multiple Machine Learning algorithms are trained and evaluated, including:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

The models are compared using performance metrics, and XGBoost is selected as the final prediction model.

4. Model Deployment

The trained XGBoost model, along with the required preprocessing objects such as encoders and scaler, is serialized using Pickle (.pkl) files for deployment.

5. Flask Web Application

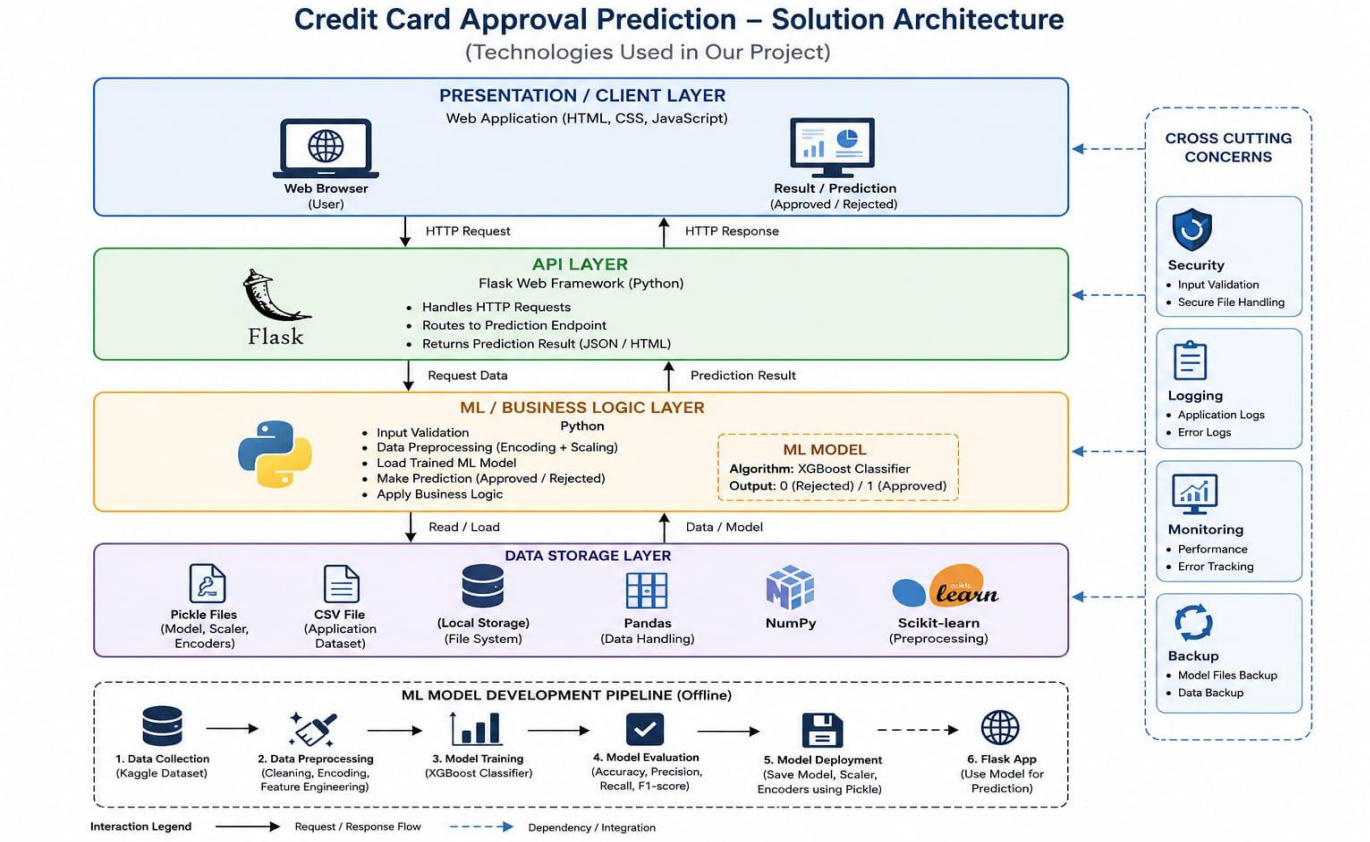
A Flask-based web application is developed to provide a simple user interface where users can enter applicant information. The application loads the trained model and predicts whether the applicant is likely to receive credit card approval.

6. Prediction Result

After processing the input data, the application displays one of the following results:

- Approved
- Rejected

The prediction is generated instantly based on the trained Machine Learning model.



Prerequisites

Before running the application, ensure the following software and libraries are installed on your system.

- Software Requirements
- Python 3.10 or above
- Visual Studio Code
- Jupyter Notebook
- GitHub Account
- Modern Web Browser (Google Chrome or Microsoft Edge)

Python Libraries

- Flask
- NumPy
- Pandas
- Scikit-learn
- XGBoost
- Matplotlib
- Seaborn
- Pickle

Hardware Requirements

- Windows 10/11 Operating System
- Minimum 4 GB RAM
- Intel Core i3 Processor or above
- At least 2 GB of free disk space

Project Workflow:

Milestone 1: Project Planning & Dataset Collection

- **Activity 1.1**: Problem Identification
- **Activity 1.2**: Dataset Collection
- **Activity 1.3**: Solution Architecture Design
- **Activity 1.4**: Development Environment Setup

Milestone 2: Data Preprocessing

- **Activity 2.1**: Data Cleaning
- **Activity 2.2**: Handling Missing Values
- **Activity 2.3**: Feature Encoding
- **Activity 2.4**: Feature Scaling

Milestone 3: Machine Learning Model Development

- **Activity 3.1**: Model Training
- **Activity 3.2**: Model Evaluation
- **Activity 3.3**: Model Comparison
- **Activity 3.4**: Final Model Selection

Milestone 4: Flask Web Application Development

- **Activity 4.1**: Flask Application Development
- **Activity 4.2**: User Interface Design
- **Activity 4.3**: Model Integration
- **Activity 4.4**: Prediction Display

Milestone 5: Testing & Deployment

- **Activity 5.1**: Functional Testing
- **Activity 5.2:** Performance Evaluation
- **Activity 5.3:**GitHub Repository

Milestone 1 Project Planning & Dataset Collection

The first milestone focused on understanding the project requirements, collecting the dataset, designing the solution architecture, and setting up the development environment. Proper planning ensured that all subsequent stages of the project could be implemented efficiently.

Activity 1.1: Problem Identification

The objective of this project is to develop an intelligent system that predicts whether a credit card application should be approved or rejected based on an applicant's demographic and financial information. Manual credit approval processes require significant time and effort and may lead to inconsistent decisions. By leveraging Machine Learning algorithms, the proposed system provides a faster, more accurate, and reliable prediction process.

Activity 1.2: Dataset Collection

The dataset for this project was collected from Kaggle. It consists of applicant information and historical credit records that are used to train and evaluate the Machine Learning models.

The dataset contains the following files:

- application_record.csv
- credit_record.csv

These files were merged and processed to create the final dataset used for model development.

Dataset Source:

https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction

Activity 1.3: Solution Architecture Design

A solution architecture was designed before implementation to define the complete workflow of the system. The architecture illustrates how applicant data flows through different stages, including data preprocessing, Machine Learning model training, model serialization, Flask application integration, and prediction generation.

The architecture provides a clear understanding of the interaction between different project components and ensures a modular and maintainable system design.

Activity 1.4: Development Environment Setup

The following tools and technologies were configured before starting the implementation:

Software / Tool	Purpose
Python 3.x	Programming Language
Jupyter Notebook	Data Analysis & Model Development
Visual Studio Code	Flask Application Development
Flask	Web Framework
GitHub	Version Control
Pandas	Data Manipulation
NumPy	Numerical Computation
Scikit-learn	Machine Learning
XGBoost	Final Prediction Model
Matplotlib & Seaborn	Data Visualization

Milestone 2: Data Preprocessing

The second milestone focused on preparing the collected dataset for Machine Learning model development. Since raw datasets often contain missing values, duplicate records, and categorical attributes, several preprocessing techniques were applied to improve the quality of the data before training the models.

Proper preprocessing ensures that the Machine Learning models receive clean, consistent, and meaningful input data, thereby improving prediction accuracy and overall model performance.

Activity 2.1: Data Understanding

Initially, the dataset was explored to understand its structure, available features, data types, and missing values. Basic statistical analysis was performed to identify inconsistencies and determine the preprocessing steps required.

The following operations were carried out:

- Loaded the dataset into Jupyter Notebook.
- Examined the number of rows and columns.
- Displayed feature names and data types.
- Checked missing values.
- Performed descriptive statistical analysis.

	ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	NAME_INCOME_TYPE
0	5008804	M	Y	Y	0	427500.0	Working
1	5008804	M	Y	Y	0	427500.0	Working
2	5008804	M	Y	Y	0	427500.0	Working
3	5008804	M	Y	Y	0	427500.0	Working
4	5008804	M	Y	Y	0	427500.0	Working

Activity 2.2: Data Cleaning

The collected dataset contained unnecessary information that could negatively affect model performance. Therefore, the dataset was cleaned before model training.

The following preprocessing tasks were performed:

- Removed duplicate records
- Removed unnecessary columns.
- Checked for inconsistent values.

- Verified missing values.

This ensured that only relevant and reliable data were used during model development.

```
# check the size of the final dataset
print(final_data.shape)

# check missing values
print(final_data.isnull().sum())

#display the first five rows
final_data.head()
```

```
(36457, 19)
ID                          0
CODE_GENDER                 0
FLAG_OWN_CAR                0
FLAG_OWN_REALTY             0
CNT_CHILDREN                0
AMT_INCOME_TOTAL            0
NAME_INCOME_TYPE            0
NAME_EDUCATION_TYPE         0
NAME_FAMILY_STATUS          0
NAME_HOUSING_TYPE           0
DAYS_BIRTH                  0
DAYS_EMPLOYED               0
FLAG_MOBIL                  0
FLAG_WORK_PHONE             0
FLAG_PHONE                  0
FLAG_EMAIL                  0
OCCUPATION_TYPE         11323
CNT_FAM_MEMBERS             0
TARGET                      0
dtype: int64
```

Activity 2.3: Feature Engineering & Encoding

The dataset contained several categorical features that could not be directly processed by Machine Learning algorithms.

The following categorical attributes were encoded:

- Gender
- Car Ownership
- Property Ownership
- Income Type
- Education Type
- Family Status
- Housing Type

- Occupation Type

Label Encoding was applied to convert these categorical values into numerical representations suitable for model training.

	ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	NAME_INCOME_TYPE	NAME_EDUC
0	5008804	1	1	1	0	427500.0	4	
1	5008805	1	1	1	0	427500.0	4	
2	5008806	1	1	1	0	112500.0	4	
3	5008808	0	0	1	0	270000.0	0	
4	5008809	0	0	1	0	270000.0	0	

Activity 2.4: Feature Scaling

The numerical features were standardized using StandardScaler to ensure that all features contributed equally during model training.

Feature scaling improves model convergence and enhances prediction performance, particularly for algorithms sensitive to feature magnitudes.

The fitted scaler was saved as a .pkl file for use during Flask application deployment.

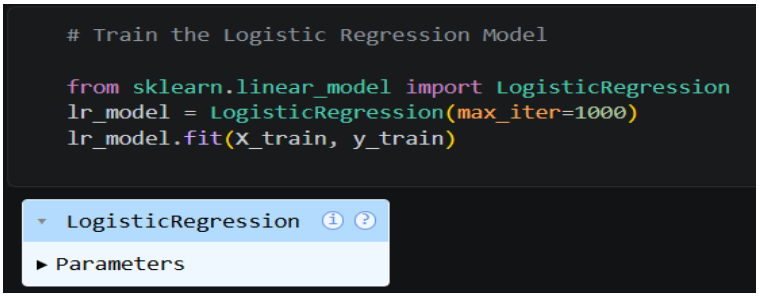
Milestone 3: Machine Learning Model Development

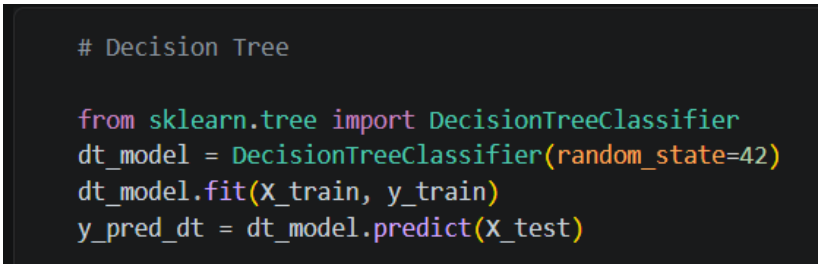
The third milestone focused on developing and evaluating multiple Machine Learning models for predicting credit card approval. The preprocessed dataset was divided into training and testing datasets, and different classification algorithms were implemented. Their performance was evaluated using appropriate metrics to identify the most suitable model for the application.

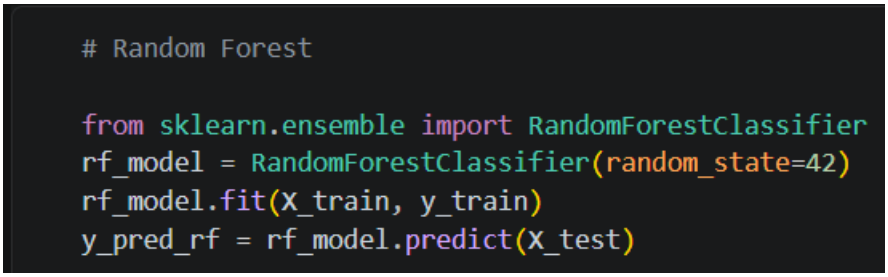
Activity 3.1: Model Training

After preprocessing, the dataset was divided into training and testing sets using the train-test split technique. Four Machine Learning classification algorithms were trained:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost
- Each model was trained using the same dataset to ensure a fair performance comparison.







```
    # Train the model

    xgb = XGBClassifier(
        random_state=42,
        eval_metric='logloss'
    )
    xgb.fit(X_train, y_train)
    y_pred_xgb = xgb.predict(X_test)
```

Activity 3.2: Model Evaluation

The trained models were evaluated using standard classification metrics. The primary evaluation metric was Accuracy, while confusion matrices and classification reports were also analyzed to assess the prediction performance.

The evaluation helped identify the strengths and weaknesses of each model.

```
Accuracy: 0.9839550191991223

Confusion Matrix:
[[7175    0]
 [ 117    0]]

Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99      7175
           1       0.00      0.00      0.00       117

    accuracy                           0.98      7292
   macro avg       0.49      0.50      0.50      7292
weighted avg       0.97      0.98      0.98      7292
```

Activity 3.3: Model Comparison

The accuracy of all four Machine Learning models was compared to determine the best-performing algorithm.

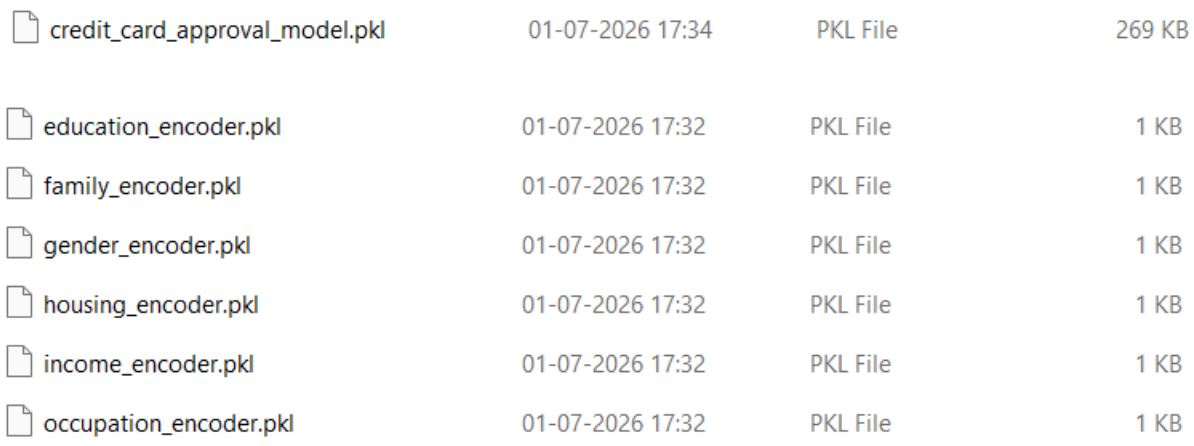
The comparison showed that XGBoost achieved the highest prediction accuracy among all the implemented models. Therefore, it was selected as the final prediction model for deployment.

```
Model Comparison
                Model  Accuracy  Precision  Recall  F1-Score
0  Logistic Regression    0.9840     0.0000  0.0000    0.0000
1        Decision Tree    0.9812     0.3333  0.1709    0.2260
2        Random Forest    0.9824     0.3778  0.1453    0.2099
3              XGBoost    0.9841     0.5238  0.0940    0.1594
```

Activity 3.4: Model Serialization

After selecting XGBoost as the final model, it was saved using the Pickle library. The preprocessing objects, including the scaler and categorical encoders, were also serialized and stored as .pkl files.

The following files were generated:



Milestone 4: Flask Web Application Development

The fourth milestone focused on developing a user-friendly web application using the Flask framework. The trained XGBoost model was integrated with the web application to allow users to enter applicant details and instantly receive a credit card approval prediction.

The application was designed with a simple interface to ensure ease of use while maintaining efficient communication between the frontend and the Machine Learning model.

Activity 4.1: Flask Application Development

The Flask framework was used to build the backend of the application. The application accepts user input through an HTML form, processes the entered values, and passes them to the trained XGBoost model for prediction.

The Flask application consists of the following files:

- app.py
- templates/index.html
- Model (.pkl) files

The backend is responsible for loading the trained model, preprocessing the input data, generating predictions, and displaying the final result.

```python
from flask import Flask, render_template, request
import pandas as pd
import pickle
import os

app = Flask(__name__)

# ------------------------------------------------
# Load Model, Scaler and Encoders
# ------------------------------------------------

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

model = pickle.load(open(os.path.join(BASE_DIR, "credit_card_approval_model.pkl"), "rb"))

print("Loading model from:", os.path.join(BASE_DIR, "credit_card_approval_model.pkl"))
print("Model expects:", model.n_features_in_)

scaler = pickle.load(open(os.path.join(BASE_DIR, "scaler.pkl"), "rb"))

gender_encoder = pickle.load(open(os.path.join(BASE_DIR, "gender_encoder.pkl"), "rb"))
car_encoder = pickle.load(open(os.path.join(BASE_DIR, "car_encoder.pkl"), "rb"))
realty_encoder = pickle.load(open(os.path.join(BASE_DIR, "realty_encoder.pkl"), "rb"))
income_encoder = pickle.load(open(os.path.join(BASE_DIR, "income_encoder.pkl"), "rb"))
education_encoder = pickle.load(open(os.path.join(BASE_DIR, "education_encoder.pkl"), "rb"))
family_encoder = pickle.load(open(os.path.join(BASE_DIR, "family_encoder.pkl"), "rb"))
housing_encoder = pickle.load(open(os.path.join(BASE_DIR, "housing_encoder.pkl"), "rb"))
occupation_encoder = pickle.load(open(os.path.join(BASE_DIR, "occupation_encoder.pkl"), "rb"))
```

```
@app.route("/")
def home():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
```

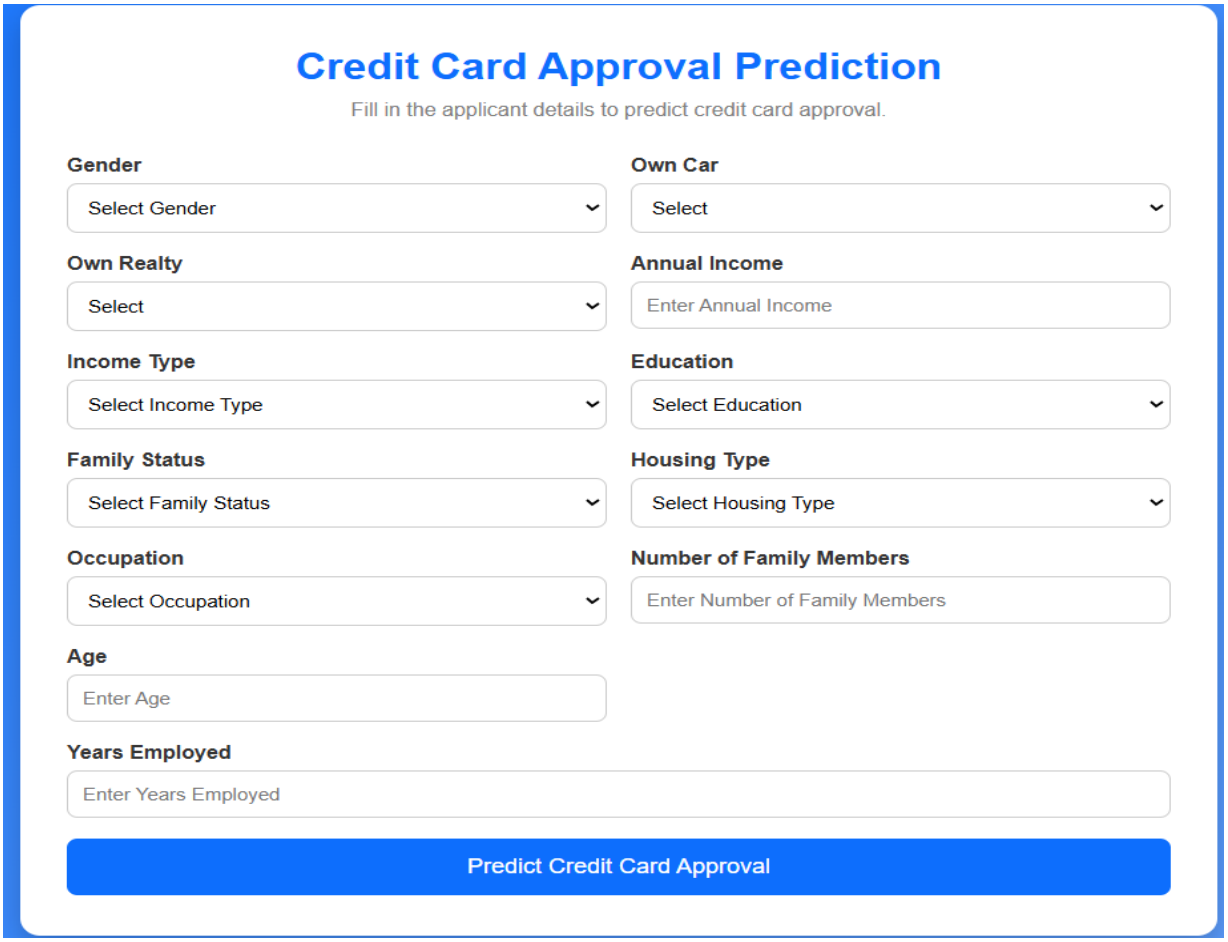
Activity 4.2: User Interface Design

A simple HTML-based user interface was developed to collect applicant information required for prediction.

The interface includes input fields such as:

- Gender
- Age
- Annual Income
- Employment Status
- Education Level
- Family Status
- Housing Type
- Occupation
- Number of Children
- Other required applicant details

The interface was designed to provide a clean and user-friendly experience.



Activity 4.3: Model Integration

The trained XGBoost model and preprocessing files were successfully integrated into the Flask application.

The prediction workflow includes:

1. User enters applicant details.
2. Flask receives the input.
3. Input data is encoded and scaled.
4. The XGBoost model processes the input.
5. The prediction result is generated.
6. The result is displayed on the web page.

This integration enables real-time prediction without retraining the model.

Activity 4.4: Prediction Result Display

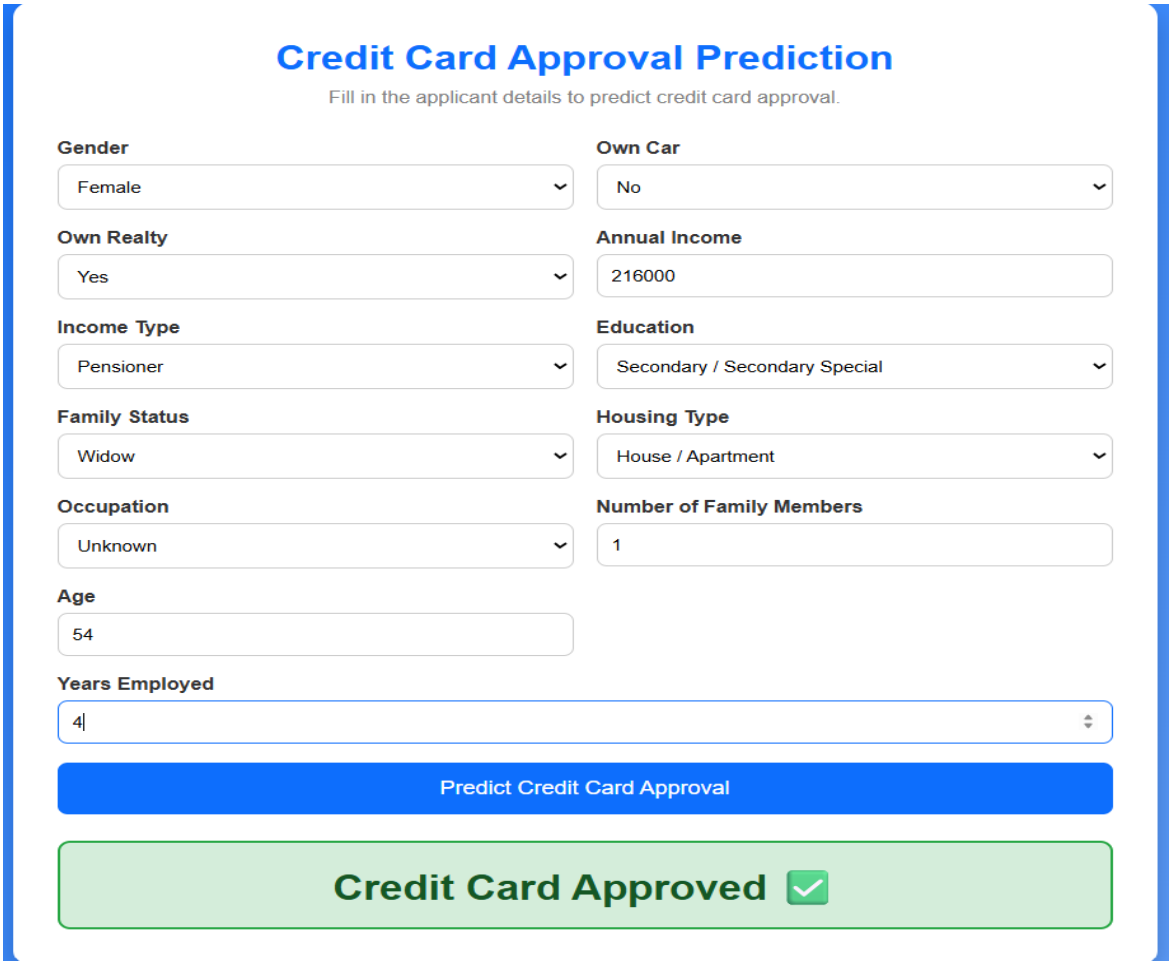
After processing the applicant information, the application displays the final prediction result.

Possible outputs include:

Credit Card Approved

Credit Card Rejected

The result is generated instantly based on the trained XGBoost model.

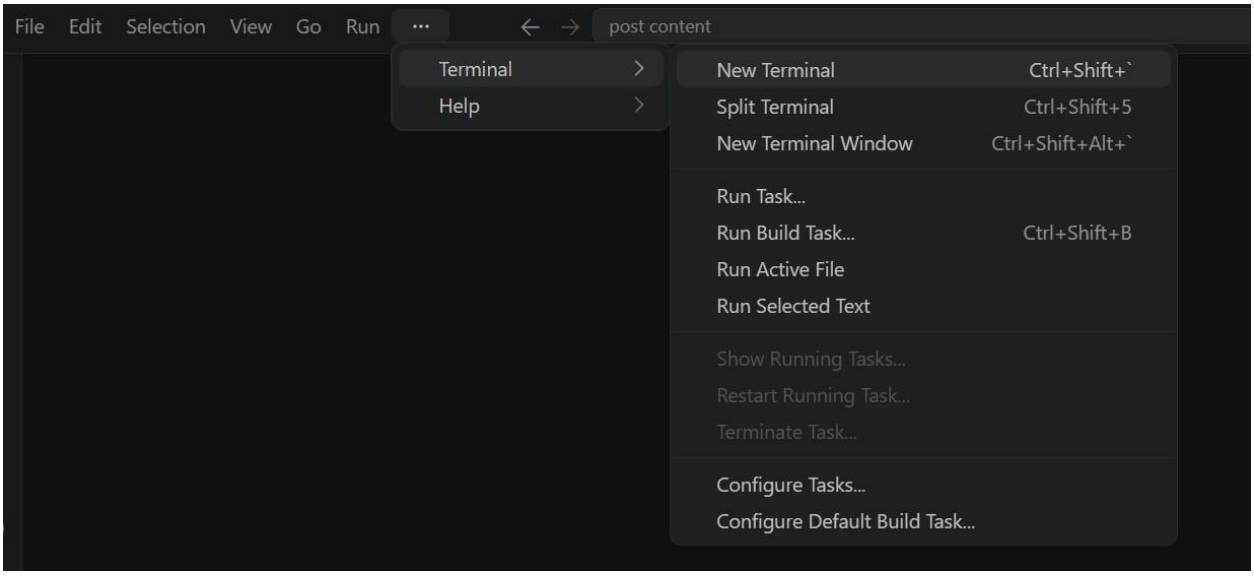


Milestone 5: Testing and Deployment

The fifth milestone focused on validating the functionality of the developed application and ensuring that the Machine Learning model produced accurate predictions. The application was tested with different applicant records to verify the correctness of the prediction process. The project was then organized and prepared for deployment and submission.

Activity 5.1:Application Testing

- Open a New Terminal



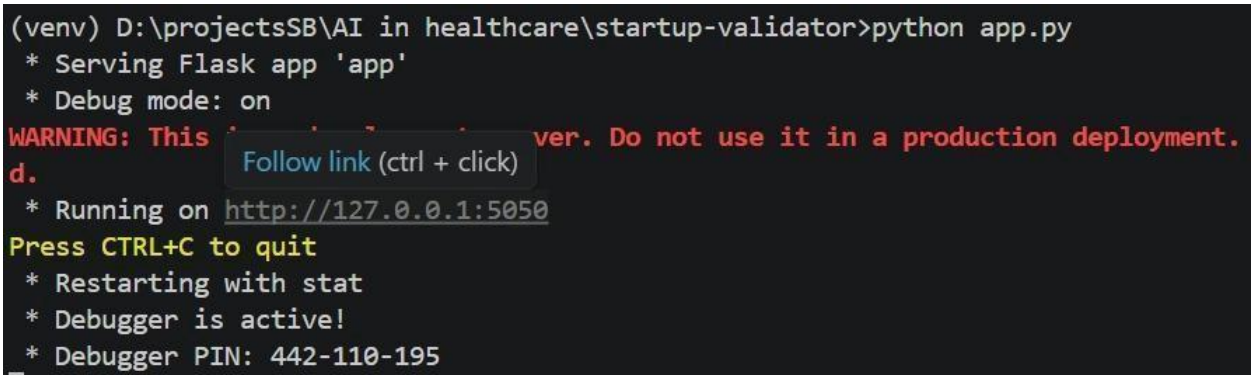
Activity 5.2: Application Testing

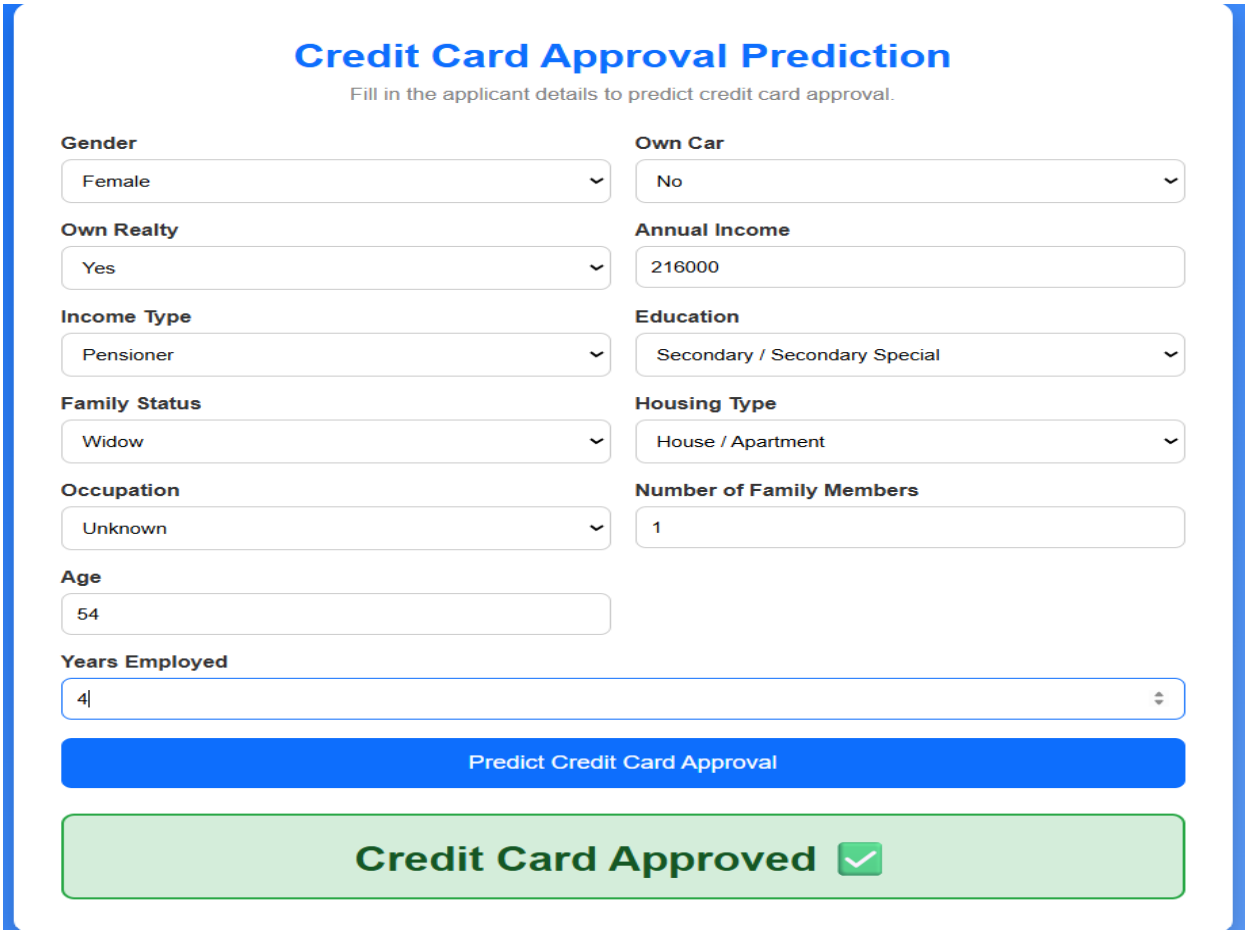
Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.





Activity 5.3: GitHub Repository Preparation

The complete project was organized and uploaded to GitHub for version control and project submission.

The repository includes:

- Source code
- Flask application
- Machine Learning model files (.pkl)
- HTML templates
- README.md
- requirements.txt
- Project documentation
- Screenshots

The dataset was excluded from the repository because it exceeded GitHub's upload size limit. The Kaggle dataset link was provided in the README file.

Exploring the Website's Web Pages:

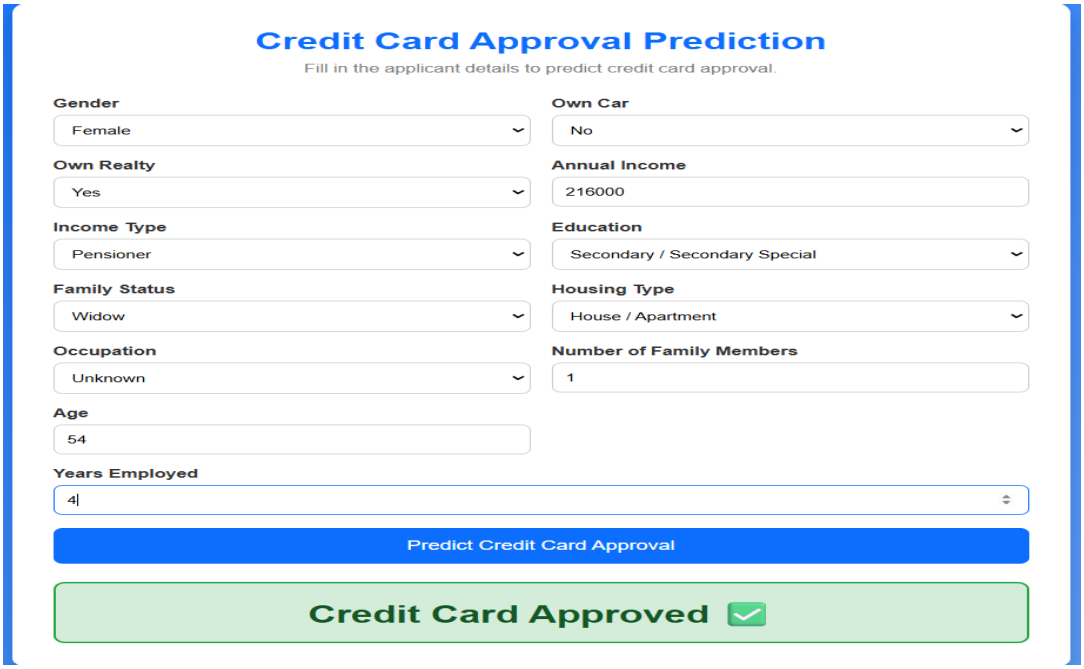
Home / Generator Page:

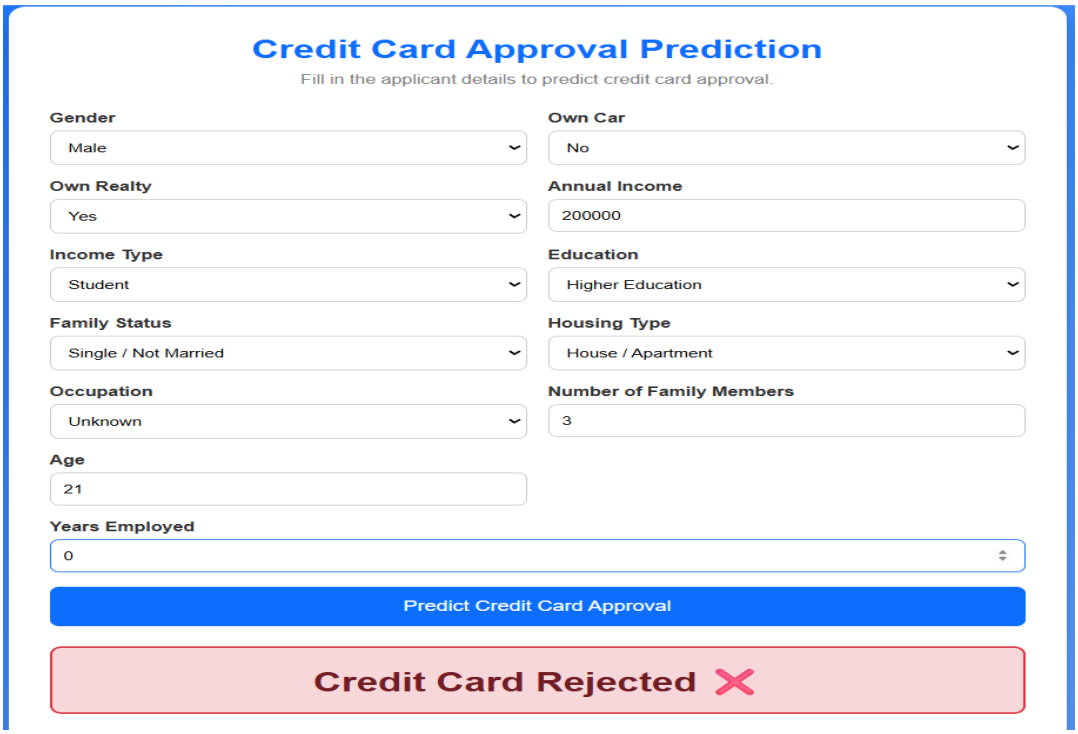
Description: The Home Page is the main interface of the Credit Card Approval Prediction application. It provides users with an easy-to-use interface to access the prediction system and enter the required applicant details.

Input Section:

Description: The Input Section allows users to enter the applicant's personal and financial information, such as gender, income, education, occupation, and family details. After filling in all the required fields, the user clicks the Predict button to submit the information for prediction.

Results Section:





Description: The Result Section displays the prediction generated by the trained XGBoost model. Based on the entered applicant details, the application shows whether the credit card application is Approved or Rejected in a clear and user-friendly format.

Conclusion:

The Credit Card Approval Prediction Using Machine Learning project successfully demonstrates how Machine Learning can be used to automate the credit card approval process. By utilizing applicant demographic and financial information, the system predicts whether a credit card application is likely to be approved or rejected with good accuracy.

The project involved data collection, preprocessing, feature engineering, model training, and performance evaluation using multiple Machine Learning algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost. After comparing the models, XGBoost was selected as the final model due to its superior performance.

A user-friendly Flask web application was developed to provide real-time predictions through a simple web interface. The trained model was successfully integrated with the application, enabling users to enter applicant details and instantly receive prediction results.

Overall, the project provides an efficient, reliable, and practical solution for credit card approval prediction. In the future, the system can be enhanced by deploying it on a cloud platform, integrating a database, implementing user authentication, and incorporating real-time financial data to further improve its scalability and prediction accuracy.